

HEITOR RODRIGUES SABINO

DETECÇÃO DE FRAUDES FINANCEIRAS: ALGORITMO NÃO SUPERVISIONADO DE AGRUPAMENTO HDBSCAN* EM COMPARAÇÃO COM ABORDAGENS SUPERVISIONADAS DE CLASSIFICAÇÃO RANDOM FOREST E XGBOOST

LAVRAS – MG

2026

HEITOR RODRIGUES SABINO

DETECÇÃO DE FRAUDES FINANCEIRAS: ALGORITMO NÃO SUPERVISIONADO DE AGRUPAMENTO HDBSCAN* EM COMPARAÇÃO COM ABORDAGENS SUPERVISIONADAS DE CLASSIFICAÇÃO RANDOM FOREST E XGBOOST

Trabalho de conclusão de curso apresentado à Universidade Federal de Lavras, como parte das exigências do curso de Ciências da Computação, para a obtenção do título de Bacharel.

Elaine Cecilia Gatto
Orientador

Natanael Fabrício Dacioli Batista
Coorientador

LAVRAS – MG

2026

HEITOR RODRIGUES SABINO

DETECÇÃO DE FRAUDES FINANCEIRAS: ALGORITMO NÃO SUPERVISIONADO DE AGRUPAMENTO HDBSCAN* EM COMPARAÇÃO COM ABORDAGENS SUPERVISIONADAS DE CLASSIFICAÇÃO RANDOM FOREST E XGBOOST

Trabalho de conclusão de curso apresentado à Universidade Federal de Lavras, como parte das exigências do curso de Ciências da Computação, para a obtenção do título de Bacharel.

APROVADA em - .

- UFLA
- UFLA

Elaine Cecilia Gatto
Orientador

Natanael Fabrício Dacioli Batista
Co-Orientador

LAVRAS – MG
2026

Dedico à minha mãe Cintia Maria da Silva e ao meu pai Humberto Alencar Sabino.

AGRADECIMENTOS

Agradeço aos meus orientadores, professores e amigos.

"Os circuitos de consagração social serão tanto mais eficazes quanto maior a distância social do objeto consagrado."
(Pierre Bourdie)

RESUMO

-

Palavras-chave: Fraudes Financeiras; Aprendizado de Máquina; HDBSCAN*; Agrupamento; Classificação.

ABSTRACT

-

Keywords: Financial Fraud; Machine Learning; HDBSCAN*; Clustering; Classification.

SUMÁRIO

1	Introdução	8
2	Trabalhos Correlatos	9
3	Fundamentação Teórica	10
3.1	Aprendizado de Máquina	10
3.1.1	Aprendizado Supervisionado	10
3.1.2	Aprendizado Não Supervisionado	11
3.1.3	Pipeline de Desenvolvimento e Validação	12
3.1.3.1	Estratificação	12
3.1.3.2	Principal Component Analysis (PCA)	12
3.1.3.3	Escalonamento e padronização	12
3.1.3.4	Validação Cruzada Aninhada	12
3.2	Algoritmos Supervisionados	14
3.2.1	Random Forest	14
3.2.2	Extreme Gradient Boosting	15
3.3	Algoritmo Não Supervisionado	16
3.3.1	DBSCAN*	16
3.3.2	HDBSCAN*	16
4	Metodologia	19
4.1	Conjunto de Dados	19
4.2	Pré-processamento	19
4.3	Parametrização e avaliação de consistência	24
4.3.1	Nested Cross-Validation	24
4.3.2	Grid Search	26
4.4	Modelos	26
4.4.1	Supervisionados	26
5	Resultados e Discussão	27
6	Conclusão	28
	REFERÊNCIAS	29

1 INTRODUÇÃO

2 TRABALHOS CORRELATOS

3 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos teóricos fundamentais que embasam o desenvolvimento deste trabalho.

3.1 Aprendizado de Máquina

O Aprendizado de Máquina (*Machine Learning*, ML) consiste no desenvolvimento de algoritmos que capacitam computadores a aprender a realizar tarefas a partir de dados, sem a necessidade de programação explícita para cada regra de execução, segundo Samuel (1959), que introduziu o tema.

Segundo Rezazadeh (2025), as principais abordagens do Aprendizado de Máquina dividem-se em quatro categorias: o aprendizado supervisionado, baseado em dados rotulados; o aprendizado não supervisionado, aplicado a dados não rotulados; o aprendizado semissupervisionado, que combina as duas abordagens anteriores ao utilizar simultaneamente dados rotulados e não rotulados; e o aprendizado por reforço, no qual o agente aprende por tentativa e erro, a partir de recompensas e penalidades obtidas na interação com o ambiente. As subseções seguintes tratam do aprendizado supervisionado e do não supervisionado, que fundamentam os modelos comparados neste trabalho.

3.1.1 Aprendizado Supervisionado

O aprendizado supervisionado destaca-se como uma das metodologias mais proeminentes dentro do Aprendizado de Máquina (BZDOK; KRZYWINSKI; ALTMAN, 2018). Essa abordagem é caracterizada por sua capacidade de aprender com experiências passadas mediante a utilização de conjuntos de dados históricos previamente rotulados. Ao analisar esses dados rotulados durante a fase de treinamento, o algoritmo adquire a capacidade de prever ou atribuir os rótulos apropriados a novos dados não classificados.

As técnicas primárias empregadas no aprendizado supervisionado são a classificação e a regressão, as quais diferem fundamentalmente na natureza da previsão alvo (BZDOK; KRZYWINSKI; ALTMAN, 2018):

- **Classificação:** É aplicada quando o vetor de saída previsto é de natureza categórica ou discreta. A função primária de um classificador é atribuir novos pontos de dados a categorias distinhas predefinidas, utilizando o conhecimento extraído da base de treinamento (GOPAL, 2018).

- **Regressão:** Ao contrário da classificação, a técnica de regressão é utilizada quando a saída desejada é de natureza numérica ou contínua, conforme contextualizado por Rezazadeh (2025).

3.1.2 Aprendizado Não Supervisionado

A segunda principal técnica em ML é o aprendizado não supervisionado, empregado na construção de modelos preditivos em situações onde não há disponibilidade de dados previamente rotulados para o treinamento (SOLORIO-FERNÁNDEZ; CARRASCO-OCHOA; MARTÍNEZ-TRINIDAD, 2020). Consequentemente, não há informações prévias sobre a correlação exata entre as características de entrada e saída.

Nesse caso, o papel do algoritmo é permitir que os dados falem por si, revelando as estruturas intrínsecas do conjunto sem a orientação de rótulos predefinidos, o que caracteriza o aprendizado não supervisionado como um processo orientado pelos dados (REZAZADEH, 2025). Uma das distinções mais significativas em relação à metodologia supervisionada é que as abordagens não supervisionadas frequentemente requerem um grau mais elevado de interpretação humana. Uma vez que não existem métricas estatísticas simples que definam um resultado de forma binária (como um rótulo "certo ou errado"), o processo baseia-se fortemente em medidas de estatística descritiva e técnicas de visualização de dados (GOPAL, 2018). As principais categorias do aprendizado não supervisionado incluem:

- **Agrupamento (Clustering):** Considerada a técnica não supervisionada mais prevalente (KONONENKO; KUKAR, 2007), o agrupamento visa organizar os dados com base em suas semelhanças intrínsecas. O objetivo é garantir que os itens dentro de um mesmo agrupamento (*cluster*) compartilhem características fortes (como formato, cor ou dimensão), enquanto exibem alta dissimilaridade em relação aos itens de outros agrupamentos (IGLESIAS; ZSEBY; ZIMEK, 2021; SHALEV-SHWARTZ; BEN-DAVID, 2014).

- **Associação:** Esta técnica foca na descoberta de dependências ou correlações entre entradas individuais de dados para, posteriormente, mapeá-las em conjunto (DIKE et al., 2018).

- **Redução de Dimensionalidade:** Uma técnica adjacente que visa simplificar a complexidade dos dados não rotulados, extraindo as características mais relevantes sem perder a representatividade da informação (GOPAL, 2018).

3.1.3 Pipeline de Desenvolvimento e Validação

A implementação de modelos de aprendizado supervisionado e não supervisionado seguem um fluxo metodológico geral, onde percorre etapas de pré-processamento até a avaliação final.

3.1.3.1 Estratificação

3.1.3.2 Principal Component Analysis (PCA)

A Análise de Componentes Principais é um procedimento de estatística multivariável que reduz a dimensionalidade do *dataset*, isso de forma a permitir de maneira intuitiva e estruturada a interpretação das principais *features* inerentes a um conjunto de dados multivariado complexo (GREENACRE et al., 2022).

3.1.3.3 Escalonamento e padronização

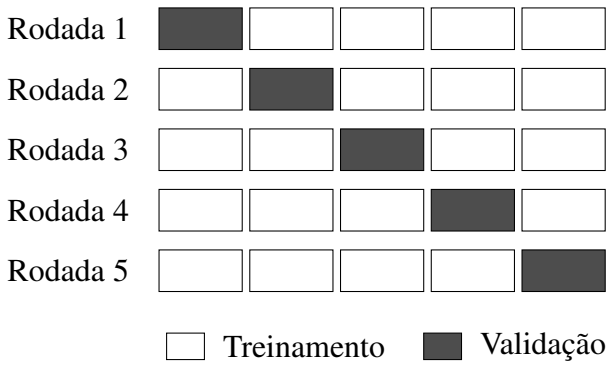
3.1.3.4 Validação Cruzada Aninhada

Com o fim de mitigar o viés de otimismo na estimativa de desempenho, assegurar a estabilidade estatística, garantindo a consistência e confiabilidade dos resultados, e, também, fazer a cobertura integral do conjunto de dados durante o processo de modelagem, adotou-se a metodologia de validação cruzada aninhada (*nested cross-validation*). Para Krstajic et al. (2014), parafraseando a literatura clássica (STONE, 1974; VARMA; SIMON, 2006), essa abordagem é indispensável quando se realiza otimização de hiperparâmetros, pois impede o

vazamento de informações (*data leakage*) ao separar de forma clara a etapa de calibração da etapa de generalização.

Segundo Berrar (2024), a validação cruzada *k-fold* consiste em particionar o conjunto de dados disponível, por amostragem aleatória sem reposição, em k subconjuntos disjuntos (*folds*) de tamanho aproximadamente igual. Em cada uma de k rodadas, o modelo é treinado com $k-1$ subconjuntos, que formam o conjunto de treinamento, e avaliado no subconjunto restante, denominado conjunto de validação. O procedimento se repete até que cada subconjunto tenha sido utilizado uma vez para validação, e a média das k medidas de desempenho constitui a estimativa obtida por validação cruzada. Quando a amostragem é estratificada, cada subconjunto preserva a proporção das classes do conjunto original. A Figura 3.1 ilustra o procedimento para $k = 5$.

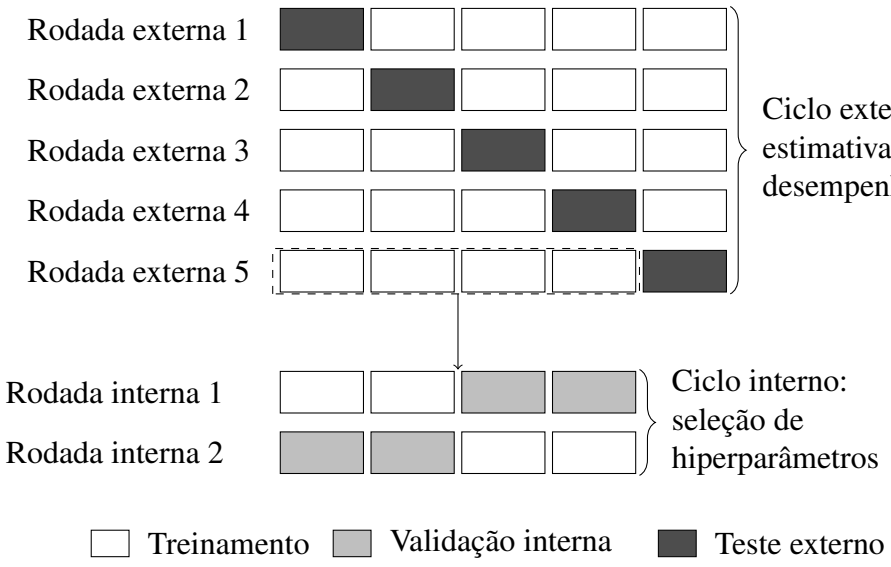
Figura 3.1 – Validação cruzada *k-fold* com $k = 5$.



Fonte: Adaptado de Berrar (2024).

A validação cruzada aninhada executa esse procedimento em dois níveis. O ciclo externo corresponde a uma validação cruzada *k-fold* convencional, destinada a estimar o desempenho de generalização. Em cada rodada externa, o conjunto de treinamento é submetido a uma nova validação cruzada, o ciclo interno, na qual são avaliadas as configurações candidatas de hiperparâmetros. A configuração de melhor desempenho médio no ciclo interno é utilizada para treinar o modelo com todo o conjunto de treinamento externo, que é então avaliado no subconjunto de teste externo. A estimativa final corresponde à média dos desempenhos obtidos nas k rodadas externas, e as configurações selecionadas podem diferir entre as rodadas. Desse modo, a seleção de hiperparâmetros permanece separada da avaliação do modelo (BERRAR, 2024). A Figura 3.2 ilustra o esquema.

Figura 3.2 – Validação cruzada aninhada com cinco rodadas externas e duas internas. O ciclo interno é representado apenas para a quinta rodada externa.



Fonte: Adaptado de Berrar (2024).

Neste trabalho, a seleção de hiperparâmetros do HDBSCAN* difere desse esquema. Cada ciclo interno indica a configuração de melhor desempenho e, dentre as configurações assim obtidas, seleciona-se a de maior desempenho, que é aplicada a todas as rodadas externas.

3.2 Algoritmos Supervisionados

Neste tópico, serão descritos os algoritmos supervisionados baseados em árvores de decisão selecionados para o embasamento comparativo do estudo.

3.2.1 Random Forest

Uma árvore de decisão é a menor unidade formadora de um modelo *Random Forest* (RF). Assim, segundo Zhang e Singer (2010), a estrutura de uma árvore de decisão é constituída por camadas, onde a quantidade total dos dados a serem analisados se encontra na raiz, e assim as estruturas subsequentes possuem menor quantidade de dados. À medida que os dados atravessam a árvore, eles passam por nós internos até serem classificados em subgrupos terminais mutuamente exclusivos, as folhas (WEST; BHATTACHARYA, 2016).

O modelo RF é construído a partir de várias árvores de decisão, com o fim de superar a instabilidade das estimativas de uma árvore isolada (BIAU; SCORNET, 2016). Nesse sentido, Zhang e Singer (2010) afirmam que, isoladamente, cada árvore não representa um bom modelo, mas a combinação delas gera ganho de valor. Assim, por ser uma junção de modelos (árvores de decisão), o *Random Forest* é categorizado como modelo de *ensemble* (SALMAN; KALAKECH; STEITI, 2024).

O método utiliza a técnica *Bagging* (*Bootstrap Aggregating*). Esse processo consiste em gerar múltiplos conjuntos de treinamento a partir da base de dados original, selecionando as amostras de forma aleatória e com reposição. Cada subconjunto treina uma árvore independente. O resultado final do modelo é obtido pela agregação das respostas de todos os chamados *base learners* (modelos menores como árvores de decisão): utiliza-se a média aritmética para tarefas de regressão ou o voto da maioria para classificação (SALMAN; KALAKECH; STEITI, 2024). Além da amostragem com reposição, a cada divisão de nó o RF considera apenas um subconjunto de variáveis sorteado entre as disponíveis (BIAU; SCORNET, 2016), o que torna as árvores mais diferentes e menos correlacionadas entre si e confere maior estabilidade à agregação (PROBST; WRIGHT; BOULESTEIX, 2019). A variância da floresta corresponde ao produto da correlação entre as árvores pela variância de uma árvore individual; por isso, a combinação de árvores pouco correlacionadas reduz a variância do modelo, que diminui à medida que o número de árvores aumenta, sem conduzir ao sobreajuste (BIAU; SCORNET, 2016).

3.2.2 Extreme Gradient Boosting

O *Extreme Gradient Boosting*, comumente abreviado como XGBoost, consiste em uma otimização avançada da arquitetura *Gradient Boosting Machine* (GBM). Assim como o seu predecessor, o XGBoost é classificado como um modelo *ensemble*, pois utiliza uma coleção de modelos preditivos individuais (como as árvores de decisão) integrados em uma estrutura única para atingir um objetivo comum.

Como o GBM atua como o alicerce teórico do XGBoost, é fundamental compreender a sua mecânica. O modelo é composto por um conjunto de preditores individuais, denominados *base learners* (WADE, 2020). Diferentemente do *Random Forest*, em que a construção das árvores ocorre de maneira estritamente independente, o diferencial do *Gradient Boosting* é o seu

aprendizado sequencial fundamentado na correção de erros. O GBM otimiza seus resultados construindo novas árvores direcionadas especificamente para corrigir as falhas mapeadas nas iterações anteriores, adicionando-as ao conjunto para complementar as estruturas já existentes (WADE, 2020; PAN, 2018).

A partir dessa base conceitual, o XGBoost surge como uma evolução de alta performance. O seu principal diferencial técnico é a exigência rigorosa para a expansão do modelo: uma nova ramificação só é construída se o ganho de eficiência gerado por essa inclusão for estritamente superior ao ganho da estrutura anterior. Consequentemente, caso o benefício da inclusão seja inferior ao da árvore que a originou, o algoritmo realiza uma poda, interrompendo o crescimento daquele nó (CHEN; GUESTRIN, 2016).

3.3 Algoritmo Não Supervisionado

3.3.1 DBSCAN*

O DBSCAN* é uma variante do DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) (ESTER et al., 1996; CAMPELLO et al., 2015) e, como ele, é um algoritmo de agrupamento que identifica padrões nos dados fundamentado na estimativa de níveis mínimos de densidade (SCHUBERT et al., 2017). O funcionamento do modelo depende da definição de dois parâmetros principais: um raio de alcance (ε) e um número mínimo de vizinhos (*minPts*). Durante a análise, um objeto é classificado como um "ponto central"(*core point*) se a quantidade de vizinhos presentes dentro do seu raio ε for igual ou superior ao limiar estabelecido por *minPts* (SCHUBERT et al., 2017; CAMPELLO et al., 2015).

A premissa fundamental do algoritmo é localizar regiões contínuas que satisfaçam essa densidade mínima, isolando-as de áreas mais esparsas do conjunto de dados (SCHUBERT et al., 2017). No DBSCAN*, os agrupamentos (*clusters*) são formados apenas por pontos centrais: dois pontos centrais contidos no raio ε um do outro são diretamente alcançáveis e pertencem ao mesmo agrupamento, que se expande de forma transitiva para englobar os demais pontos centrais alcançáveis a partir dos seus membros. Os pontos que não são centrais são rotulados como ruído (*noise*) (CAMPELLO et al., 2015).

3.3.2 HDBSCAN*

O HDBSCAN* (*Hierarchical DBSCAN**) é um algoritmo hierárquico de agrupamento baseado em densidade que aprimora o método de agrupamento por densidade, o DBSCAN* (CAMPELLO et al., 2015). Ao contrário do DBSCAN*, que exige a definição prévia do raio ε, o HDBSCAN* tem como parâmetro principal o número mínimo de pontos, m_{pts}, equivalente ao m_{pts} do DBSCAN* e tratado como um fator de suavização das estimativas de densidade (CAMPELLO et al., 2015).

A principal diferença do HDBSCAN* em relação ao DBSCAN* está no tratamento do raio. Em vez de fixar um valor de ε, o HDBSCAN* calcula, para cada ponto x_p, a distância núcleo $d_{core}(x_p)$, definida como a distância de x_p ao seu m_{pts}-ésimo vizinho mais próximo, contando o próprio x_p. A distância núcleo é, portanto, o menor raio ε para o qual x_p satisfaz a condição de ponto central (CAMPELLO et al., 2015), e é intuitivo que ela cresça à medida que a vizinhança do ponto se torna mais esparsa. A partir dela, define-se a distância de alcançabilidade mútua entre dois pontos x_p e x_q (Equação 3.1), que corresponde ao menor raio ε para o qual os dois pontos são alcançáveis um a partir do outro (CAMPELLO et al., 2015):

$$d_{mreach}(x_p, x_q) = \max\{d_{core}(x_p), d_{core}(x_q), d(x_p, x_q)\} \tag{3.1}$$

em que $d(x_p, x_q)$ é a distância original entre os dois pontos.

Essas distâncias definem o grafo de alcançabilidade mútua, um grafo completo cujos vértices são os pontos do conjunto de dados e cujas arestas têm como peso a distância de alcançabilidade mútua entre eles (CAMPELLO et al., 2015). Esse grafo é conceitual e não precisa ser materializado, pois os pesos das arestas são calculados sob demanda para a obtenção da sua árvore geradora mínima (NETO et al., 2022). As arestas dessa árvore são então removidas em ordem decrescente de peso, o que equivale a reduzir progressivamente o raio ε: a cada remoção, os componentes conectados de pontos centrais formam os agrupamentos, e os demais pontos passam a ser ruído (CAMPELLO et al., 2015). Dessa forma, enquanto o DBSCAN* produz um único agrupamento para o raio escolhido, o HDBSCAN* produz uma hierarquia que reúne todos os agrupamentos que o DBSCAN* obteria para o valor de m_{pts} e para qualquer $\varepsilon \in [0, \infty)$ (CAMPELLO et al., 2015).

Para obter uma partição única, os agrupamentos são extraídos da hierarquia por um critério de estabilidade, segundo o qual os agrupamentos mais proeminentes tendem a persistir por uma faixa maior de densidades (CAMPELLO et al., 2013; CAMPELLO et al., 2015). Nesse processo, componentes com menos pontos que um tamanho mínimo de agrupamento, m_{clSize}, são considerados espúrios e rotulados como ruído (CAMPELLO et al., 2015).

O valor de m_{pts} continua a influenciar o resultado. Valores maiores exigem mais vizinhos para que um ponto seja central, o que aumenta as distâncias núcleo e tende a formar agrupamentos mais suaves e possivelmente maiores, enquanto valores menores permitem agrupamentos mais detalhados e menores (GHOSH et al., 2024). Assim, diferentes faixas de m_{pts} podem levar a resultados distintos (NETO et al., 2021; NETO et al., 2022), o que torna necessária a busca por esse parâmetro.

Na detecção não supervisionada de anomalias, parte-se da premissa de que as anomalias se encontram em regiões de baixa densidade do conjunto de referência (GHOSH et al., 2024). Um método global clássico é o dos k vizinhos mais próximos (*k-Nearest Neighbors*, kNN), em que o escore de anomalia de um ponto é a distância ao seu k-ésimo vizinho mais próximo (RAMASWAMY; RASTOGI; SHIM, 2000), grandeza análoga à distância núcleo do HDBSCAN* (CAMPELLO et al., 2015).

Neste estudo, as fraudes não são identificadas apenas como os pontos rotulados como ruído. Cada transação recebe um escore de anomalia que combina a sua pertinência ao agrupamento atribuído pelo HDBSCAN* e a distância ao seu m_{pts}-ésimo vizinho mais próximo, e as transações de maior escore são sinalizadas como suspeitas.

4 METODOLOGIA

4.1 Conjunto de Dados

Os dados referentes a transações financeiras reais são sigilosos em virtude da natureza comprometedora do dado e por ser de domínio privado, visto isso, pesquisas acerca de investigações de crimes financeiros enfrentam tal barreira. Nesse sentido, o conjunto de dados utilizado possui informações limitadas acerca das colunas com o objetivo de sanar a questão da confidencialidade dos registros.

Assim, o conjunto de dados manipulado neste trabalho foi disponibilizado pelo Machine Learning Group da Université Libre de Bruxelles, ao qual está acessível pelo site Kaggle (Machine Learning Group - Université Libre de Bruxelles, 2018).

O *dataset* contém transações em cartões de crédito em setembro de 2013, feitas por titulares de cartões europeus e foram coletadas e analisadas durante uma pesquisa em colaboração com a Worldline, uma empresa francesa de serviços de pagamento e transações, fundada em 1974. Foram capturadas 284.807 transações ao longo de um período de dois dias, onde apenas 492 são classificadas como fraudulentas. Logo, o conjunto de dados é altamente desbalanceado considerando a elevada desproporção entre as classes, com a classe positiva (fraude) representando 0.172

Em relação às *features*, o *dataset* possui 31 variáveis numéricas das quais 28 (V1, V2, ..., V28) delas foram submetidas ao processo de Análise de Componente Principal (PCA), uma técnica padrão e amplamente utilizada para reduzir a dimensionalidade de tais conjuntos de dados, assim aumentando a interpretabilidade e, de forma paralela, minimizando a perda de informação (ABDI; WILLIAMS, 2010). Isso acontece por meio da criação de novas variáveis não correlacionadas que maximizam sucessivamente a variância. Concomitantemente, o PCA colabora para a anonimização dos dados (ALARFAJ et al., 2022). As *features* que não foram submetidas ao PCA são *amount*, *class* e *time*.

Tabela 4.1 – Descrição das colunas do conjunto de dados.

Variável	Descrição	Tipo
Time	Número de segundos decorridos entre a transação atual e a primeira transação registrada no conjunto de dados.	Contínua
V1 a V28	Componentes principais resultantes de uma transformação de redução de dimensionalidade (PCA). Os dados originais não são fornecidos para proteger a identidade dos usuários e informações sensíveis.	Contínua
Amount	Valor monetário da transação.	Contínua
Class	Variável alvo (rótulo). Assume o valor 1 caso a transação seja uma fraude e 0 caso seja uma transação genuína.	Binária

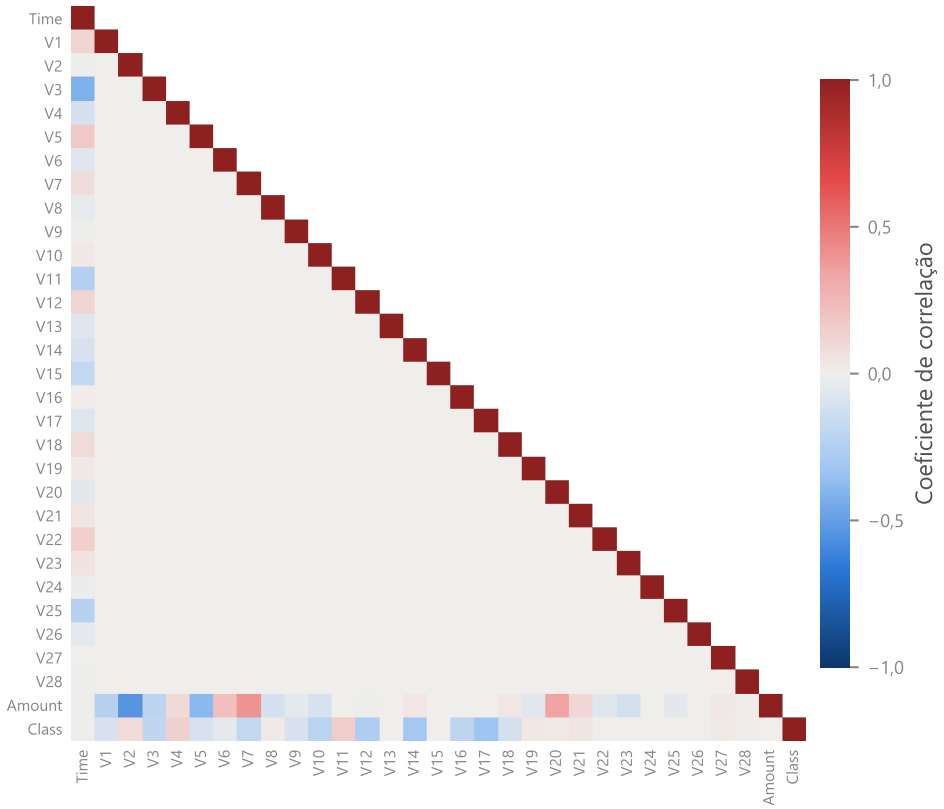
4.2 Pré-processamento

Através de processos de análise exploratória acerca da base, notou-se que não há necessidade de tratamentos quanto a dados faltantes ou nulos. O conjunto de dados foi originalmente validado nesse sentido. A base contém, no entanto, 1.081 transações que repetem outra em todas as variáveis. Elas são retiradas do conjunto de treinamento do HDBSCAN*, pois cópias de um mesmo ponto ficam a distância zero umas das outras e aumentariam artificialmente a densidade estimada na sua região; os conjuntos de validação e de teste permanecem completos, para que a avaliação cubra todas as transações.

A Figura 4.1 mostra a correlação de Pearson entre as variáveis: as componentes V1 a V28 não são correlacionadas entre si, por construção do PCA (ABDI; WILLIAMS, 2010), e as correlações restantes envolvem Time e Amount. Por conta da anonimização advinda do PCA, não é possível interpretar o que essas correlações representam, e nenhuma variável foi retirada ou fundida por causa delas: todas são mantidas nos modelos com suas devidas transformações e padronizações.

A variável Time registra os segundos decorridos desde a primeira transação da base e cresce ao longo dos dois dias de coleta, de modo que, em seu valor bruto, apenas ordena as transações, como um identificador. A Figura 4.2 compara o horário das transações legítimas e das fraudulentas: convertidas em hora do dia, as fraudes se concentram em determinadas faixas de horário (painel b), e a faixa de maior risco tem 5,6 vezes mais fraudes por transação que a de

Figura 4.1 – Correlação de Pearson entre as variáveis da base.

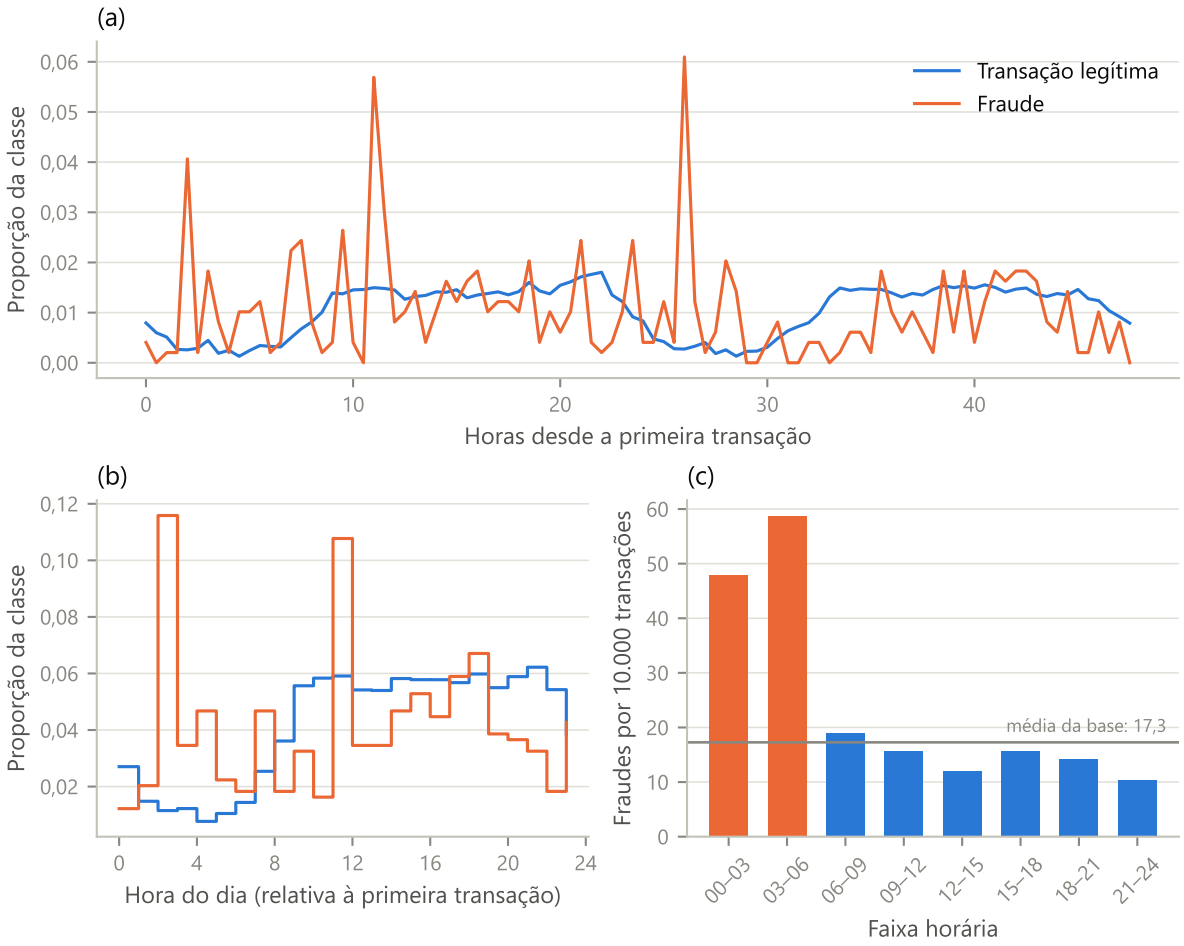


Fonte: Do autor (2026).

menor risco (painel c). Por isso, o valor bruto de Time não é usado por nenhum dos modelos. Dele se extrai apenas a posição da transação no ciclo de 24 horas, cuja distribuição difere entre fraudes e transações legítimas, como confirma o teste de Kolmogorov-Smirnov sobre a hora do dia ($D = 0{,}197$; $p < 0{,}001$). O valor de D é a maior diferença entre as proporções acumuladas das duas classes ao longo do dia: em certo horário, a fração das fraudes ocorridas até ali difere em 19,7 pontos percentuais da fração das transações legítimas. O valor de p indica que uma diferença desse tamanho seria muito improvável se as duas classes tivessem a mesma distribuição de horários.

Como a base não informa o horário da primeira transação, a hora do dia é contada a partir desse instante, e não do horário de fato. O deslocamento é o mesmo para todas as transações, de modo que a duração do ciclo, a distância entre os horários e a diferença entre

Figura 4.2 – Horário das transações legítimas e fraudulentas. (a) Distribuição ao longo da coleta; (b) distribuição por hora do dia; (c) fraudes por 10.000 transações em cada faixa de três horas.



Fonte: Do autor (2026).

fraudes e transações legítimas se preservam; apenas os rótulos das faixas da Figura 4.2 deixam de corresponder ao horário local.

A coluna Time passou então pela transformação para representar a hora do dia. O HDBSCAN*, o *Random Forest* e o XGBoost recebem essa mesma representação.

Durante a etapa de pré-processamento foram feitas algumas transformações quanto aos dados. Assim, foram feitas padronizações nas variáveis exceto pela variável Class e pelas que passaram pelo processo de PCA, pois pressupõe-se que em ordem de aplicar a transformação PCA, é necessário que passe pelo processo de padronização antes, visto que o método é sensível às escalas.

Já quanto à padronização das variáveis Amount e Time (após a transformação), foram escalonadas, pois, em ML, esse processo transforma os valores das variáveis de acordo com uma regra definida, assim essas *features* escalonadas têm o mesmo grau de influência e esse método é imune à escolha das unidades (HUANG; LI; XIE, 2015).

Para a hora do dia e para a coluna Amount foi feito o escalonamento padrão, *Z-score standardization* (Equação 4.1):

$$z = \frac{x - \mu_X}{\sigma_X} \tag{4.1}$$

em que μ_X e σ_X são a média e o desvio padrão da variável em questão.

Antes do escalonamento, a variável Amount foi transformada pelo logaritmo (Equação 4.2), uma vez que dados financeiros apresentam alta assimetria e presença severa de *outliers* (valores transacionais atípicos):

$$x' = \ln(1 + x) \tag{4.2}$$

em que x é o valor da transação. A soma de 1 mantém a transformação definida quando o valor é zero.

A assimetria foi medida pelo coeficiente tradicional de assimetria amostral (Coeficiente de Assimetria de Fisher, ou Coeficiente de Momento de Assimetria), g_1 (JOANES; GILL, 1998) (Equação 4.3):

$$g_1 = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^3}{\left(\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2\right)^{3/2}} \tag{4.3}$$

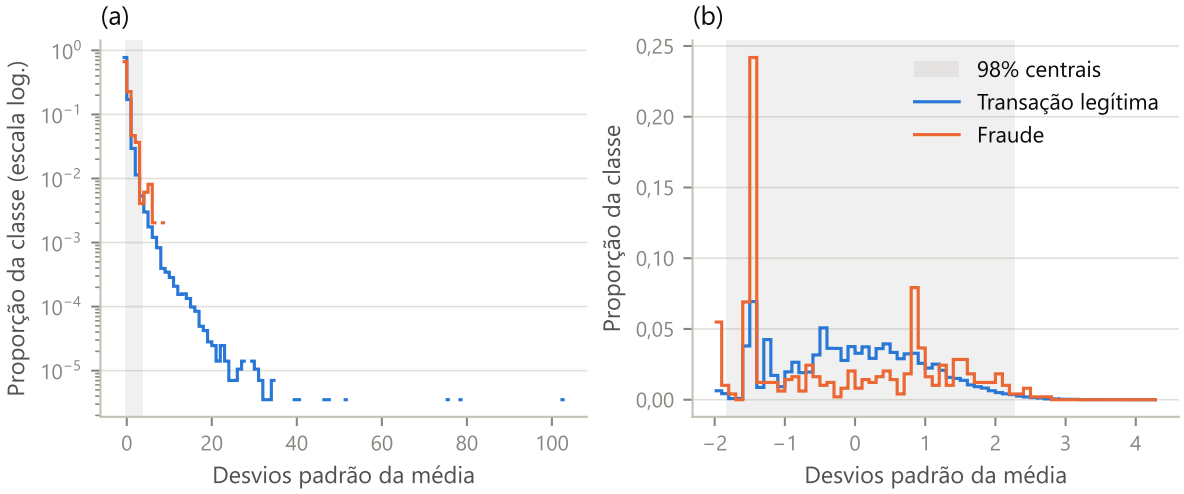
em que n é o número de transações e $\bar{x}$ é a média dos valores. O coeficiente é nulo para uma distribuição simétrica e positivo quando há uma cauda longa à direita. Em Amount, ele vale 16,98, pois a mediana é de 22,00, mas o maior valor chega a 25.691,16. Após o logaritmo, o coeficiente cai para 0,16.

Sem o logaritmo, a cauda inflaria o desvio padrão usado no escalonamento. Assim, 98% das transações ficariam entre $-0{,}35$ e 3,72 desvios padrão da média, enquanto a maior delas ficaria a 102. Como a distância euclidiana soma as diferenças ao quadrado de cada variável,

Amount sozinha decidiria os vizinhos das transações extremas e quase não diferenciaria as demais, o que contraria o objetivo do escalonamento, citado anteriormente, de dar às variáveis o mesmo grau de influência. Com o logaritmo, 98% das transações ficam entre $-1{,}83$ e 2,28 desvios padrão, e a mais extrema, a 4,2. A Figura 4.3 compara as duas distribuições.

O logaritmo preserva a ordem dos valores, de modo que a maior transação continua a mais distante da média. O que muda é a medida da diferença, que passa a ser relativa, de modo que, exemplificando, ir de 10 a 100 equivale a ir de 1.000 a 10.000. Além disso, os valores extremos da base não são fraudes. Nenhuma das 284 transações acima de 3.000 é fraudulenta, e a maior fraude é de 2.125,87. As consequências disso para os modelos são discutidas no Capítulo 5.

Figura 4.3 – Distribuição de Amount padronizada pelo Z-score, (a) sem e (b) com o logaritmo. A faixa cinza contém 98% das transações. Na figura, a média e o desvio padrão são os da base completa, e nos modelos vêm do conjunto de treinamento de cada fold.



Fonte: Do autor (2026).

4.3 Parametrização e avaliação de consistência

Um passo importante no que tange à parametrização e validação da consistência do modelo é a estratégia a ser tomada para tal feito. Visto isso, visamos realizar validação cruzada aninhada juntamente com a busca em grade como estratégia.

4.3.1 Nested Cross-Validation

O procedimento operacional se estrutura a partir de dois ciclos de repetição interdependentes: Ciclo externo (*Outer Loop*) e Ciclo Interno (*Inner Loop*).

No externo, o conjunto de dados global é particionado/replicado de maneira pseudo-aleatória em k subconjuntos (são denominados *folds*) mutuamente exclusivos e de dimensões aproximadamente equivalentes. Tanto a replicação quanto as divisões em particionamento de treino e teste são realizadas de forma estratificada, assegurando que cada *fold* preserve a proporção original da variável alvo (classes lícita e fraudulenta). De acordo com Kohavi (1995), a amostragem estratificada é superior ao esquema regular, pois reduz de forma consistente tanto o viés quanto a variância das estimativas de desempenho em conjuntos de dados do mundo real.

Já no interno, os dados de treinamento provenientes do ciclo externo são novamente subdivididos em novos subconjuntos, treino e validação. É dentro deste ciclo secundário que se executa a busca em grade (*Grid Search*) para a calibração dos hiperparâmetros do algoritmo. O modelo é treinado e validado em partições internas isoladas, mapeando-se a configuração paramétrica que otimiza a métrica alvo.

Sumariamente, a relevância fundamental da validação cruzada aninhada está no fato de que as instâncias usadas na escolha dos parâmetros ideais, o ciclo interno, jamais coincidem com os dados empregados para inferir a capacidade real preditiva do modelo, o ciclo externo. Desse modo, o método assegura a cobertura da base, analisando todas as transações de dados de forma independente, assim resultando em uma estimativa de erro metodologicamente mais robusta, imparcial e menos propensa ao *overfitting*.

Para detalhar a distribuição dos dados, este trabalho adotou uma divisão em 5 *folds* externos, compostos por subconjuntos de treino e teste. Essa partição seguiu a proporção de 80% dos dados originais para o treinamento e 20% para o teste. Em valores absolutos, cada *fold* contém aproximadamente 227.845 instâncias para treino e 56.961 para teste. Adicionalmente, a partir de cada conjunto de treinamento dos *folds* externos, realizou-se uma nova subdivisão interna para validação. Como resultado, o conjunto de treinamento final de cada *fold* passou a contar com cerca de 182.276 instâncias, enquanto o conjunto de validação reuniu 45.569 instâncias.

Conforme mencionado previamente, todas as partições foram geradas de forma estratificada, garantindo a manutenção da mesma proporção de fraudes presente na base original.

4.3.2 Grid Search

Para que um algoritmo de aprendizado de máquina extraia o máximo de sua capacidade preditiva e se adapte à topologia intrínseca do conjunto de dados, é imperativo que seus hiperparâmetros sejam calibrados. Conforme evidenciado pela literatura recente (SHAMS et al., 2023), a utilização de configurações padrão ou arbitrárias frequentemente resulta em desempenho subótimo, tornando a etapa de otimização paramétrica um requisito fundamental para a robustez do modelo.

Neste trabalho, adotou-se a Busca em Grade (*Grid Search*) como mecanismo de otimização dentro do ciclo interno da validação aninhada. O *Grid Search* é um método de varredura exaustiva que constrói uma malha multidimensional contendo todos os valores predefinidos para os parâmetros do algoritmo. Durante a fase de calibração, o modelo é instanciado, treinado e avaliado iterativamente para cada combinação única presente nesta grade (SHAMS et al., 2023).

De acordo com estudos comparativos em otimização de hiperparâmetros (LIASHCHYNSKYI; LIASHCHYNSKYI, 2019), o *Grid Search* demonstra-se a abordagem mais adequada e rigorosa para espaços de busca de baixa e média dimensionalidade. Embora possua um custo computacional linearmente proporcional ao tamanho da grade, o método assegura a exploração completa do domínio de parâmetros estipulados, garantindo a localização da combinação que matematicamente maximiza a métrica de desempenho na partição de validação, sem o risco de convergência prematura para ótimos locais.

4.4 Modelos

Os modelos analisados neste trabalho são: de natureza Supervisionada, Random Forest e XGBoost, e de natureza Não Supervisionada, HDBSCAN*.

4.4.1 Supervisionados

5 RESULTADOS E DISCUSSÃO

6 CONCLUSÃO

REFERÊNCIAS

ABDI, H.; WILLIAMS, L. J. Principal component analysis. **Wiley Interdiscip. Rev. Comput. Stat.**, Wiley, v. 2, n. 4, p. 433–459, jul. 2010.

ALARFAJ, F. K. et al. Credit card fraud detection using state-of-the-art machine learning and deep learning algorithms. **IEEE Access**, Institute of Electrical and Electronics Engineers (IEEE), v. 10, p. 39700–39715, 2022.

BERRAR, D. **Cross-validation**. 2024. *Preprint* submetido à Encyclopedia of Bioinformatics and Computational Biology, 2. ed., Elsevier.

BIAU, G.; SCORNET, E. A random forest guided tour. **TEST**, v. 25, n. 2, p. 197–227, jun. 2016.

BZDOK, D.; KRZYWINSKI, M.; ALTMAN, N. Machine learning: supervised methods. **Nature Methods**, v. 15, n. 1, p. 5–6, 2018. ISSN 1548-7105. Disponível em: <https://doi.org/10.1038/nmeth.4551>.

CAMPELLO, R. J. G. B. et al. A framework for semi-supervised and unsupervised optimal extraction of clusters from hierarchies. **Data Mining and Knowledge Discovery**, v. 27, n. 3, p. 344–371, 2013.

CAMPELLO, R. J. G. B. et al. Hierarchical density estimates for data clustering, visualization, and outlier detection. **ACM Transactions on Knowledge Discovery from Data**, v. 10, n. 1, p. 1–51, jul. 2015.

CHEN, T.; GUESTRIN, C. XGBoost: A scalable tree boosting system. **arXiv preprint arXiv:1603.02754**, mar. 2016.

DIKE, H. U. et al. Unsupervised learning based on artificial neural network: A review. In: **2018 IEEE International Conference on Cyborg and Bionic Systems (CBS)**. [S.l.: s.n.], 2018. p. 322–327.

ESTER, M. et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In: **Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)**. [S.l.: s.n.], 1996. p. 226–231.

GHOSH, K. et al. **Unsupervised Parameter-free Outlier Detection using HDBSCAN* Outlier Profiles**. 2024. *Preprint* arXiv:2411.08867.

GOPAL, M. **Applied Machine Learning**. McGraw Hill Education (India) Private Limited, 2018. ISBN 9789353160265. Disponível em: <https://books.google.com.br/books?id=BsKlDwAAQBAJ>.

GREENACRE, M. et al. Principal component analysis. **Nat. Rev. Methods Primers**, Springer Science and Business Media LLC, v. 2, n. 1, dez. 2022.

HUANG, J.; LI, Y.-F.; XIE, M. An empirical analysis of data preprocessing for machine learning-based software cost estimation. **Inf. Softw. Technol.**, Elsevier BV, v. 67, p. 108–127, nov. 2015.

IGLESIAS, F.; ZSEBY, T.; ZIMEK, A. Clustering refinement. **International Journal of Data Science and Analytics**, Springer, v. 12, n. 4, p. 333–353, 2021.

JOANES, D. N.; GILL, C. A. Comparing measures of sample skewness and kurtosis. **Journal of the Royal Statistical Society: Series D (The Statistician)**, v. 47, n. 1, p. 183–189, 1998.

KOHAVI, R. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: **Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2**. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1995. (IJCAI'95), p. 1137–1143.

KONONENKO, I.; KUKAR, M. **Machine Learning and Data Mining**. [S.l.]: Elsevier, 2007.

KRSTAJIC, D. et al. Cross-validation pitfalls when selecting and assessing regression and classification models. **J. Cheminform.**, Springer Science and Business Media LLC, v. 6, n. 1, p. 10, mar. 2014.

LIASHCHYNSKYI, P.; LIASHCHYNSKYI, P. Grid search, random search, genetic algorithm: A big comparison for nas. **arXiv preprint arXiv:1912.06059**, dez. 2019.

Machine Learning Group - Université Libre de Bruxelles. **Credit Card Fraud Detection**. Kaggle, 2018. Disponível em: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>. Acesso em: 02 jun. 2026.

NETO, A. C. A. et al. CORE-SG: Efficient computation of multiple MSTs for density-based methods. In: **Proceedings of the 38th IEEE International Conference on Data Engineering (ICDE)**. [S.l.: s.n.], 2022. p. 951–964.

NETO, A. C. A. et al. Efficient computation and visualization of multiple density-based clustering hierarchies. **IEEE Transactions on Knowledge and Data Engineering**, v. 33, n. 8, p. 3075–3089, ago. 2021.

PAN, B. Application of XGBoost algorithm in hourly PM2.5 concentration prediction. **IOP Conf. Ser. Earth Environ. Sci.**, IOP Publishing, v. 113, p. 012127, fev. 2018.

PROBST, P.; WRIGHT, M. N.; BOULESTEIX, A.-L. Hyperparameters and tuning strategies for random forest. **WIREs Data Mining and Knowledge Discovery**, p. e1301, 2019.

RAMASWAMY, S.; RASTOGI, R.; SHIM, K. Efficient algorithms for mining outliers from large data sets. In: **Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data**. [S.l.: s.n.], 2000. p. 427–438.

REZAZADEH, S. Review of machine learning. **TMP Universal Journal of Research and Review Archives**, v. 4, n. 2s, Jun 2025. Disponível em: <https://tmp.twistingmemoirs.com/index.php/ujrra/article/view/190>.

SALMAN, H. A.; KALAKECH, A.; STEITI, A. Random forest algorithm overview. **Babylonian Journal of Machine Learning**, Mesopotamian Academic Press, v. 2024, p. 69–79, jun. 2024.

SAMUEL, A. L. Some studies in machine learning using the game of checkers. **IBM J. Res. Dev.**, IBM Corp., USA, v. 3, n. 3, p. 210–229, jul. 1959. ISSN 0018-8646. Disponível em: <https://doi.org/10.1147/rd.33.0210>.

SCHUBERT, E. et al. DBSCAN revisited, revisited. **ACM Trans. Database Syst.**, Association for Computing Machinery (ACM), v. 42, n. 3, p. 1–21, set. 2017.

SHALEV-SHWARTZ, S.; BEN-DAVID, S. **Understanding machine learning**. Cambridge, England: Cambridge University Press, 2014.

SHAMS, M. Y. et al. Water quality prediction using machine learning models based on grid search method. **Multimed. Tools Appl.**, Springer Science and Business Media LLC, v. 83, n. 12, p. 35307–35334, set. 2023.

SOLORIO-FERNÁNDEZ, S.; CARRASCO-OCHOA, J. A.; MARTÍNEZ-TRINIDAD, J. F. A review of unsupervised feature selection methods. **Artificial Intelligence Review**, v. 53, n. 2, p. 907–948, 2020. ISSN 1573-7462. Disponível em: <https://doi.org/10.1007/s10462-019-09682-y>.

STONE, M. Cross-validation and multinomial prediction. **Biometrika**, JSTOR, p. 509–515, 1974.

VARMA, S.; SIMON, R. Bias in error estimation when using cross-validation for model selection. **BMC bioinformatics**, Springer, v. 7, n. 1, p. 91, 2006.

WADE, C. **Hands-On Gradient Boosting with XGBoost and scikit-learn**. Birmingham, England: Packt Publishing, 2020.

WEST, J.; BHATTACHARYA, M. Intelligent financial fraud detection: A comprehensive review. **Comput. Secur.**, Elsevier BV, v. 57, p. 47–66, mar. 2016.

ZHANG, H.; SINGER, B. H. **Recursive partitioning and applications**. 2. ed. New York, NY: Springer, 2010. (Springer Series in Statistics).